

LEARNING OBJECTIVES

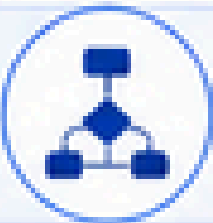
Setelah mempelajari materi ini, mahasiswa mampu:

01



Memahami konsep **White Box Testing**

02



Menjelaskan **Basis Path Testing**

03



Menghitung **Cyclomatic Complexity**

04



Mengidentifikasi jalur independen (*independent paths*)

05



Mendesain **test case** berbasis *path*



Tujuan pembelajaran ini akan membantu Anda memahami teknik pengujian struktural secara lebih logis, sistematis, dan terukur.

PENDAHULUAN WHITE BOX TESTING



Pengujian berbasis **struktur internal program (kode)**

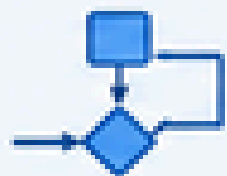


FOKUS PADA:



LOGIC

Memeriksa kebenaran logika program



CONTROL FLOW

Memastikan semua alur kendali dieksekusi



STRUKTUR PROGRAM

Menguji struktur dan keterkaitan antar komponen

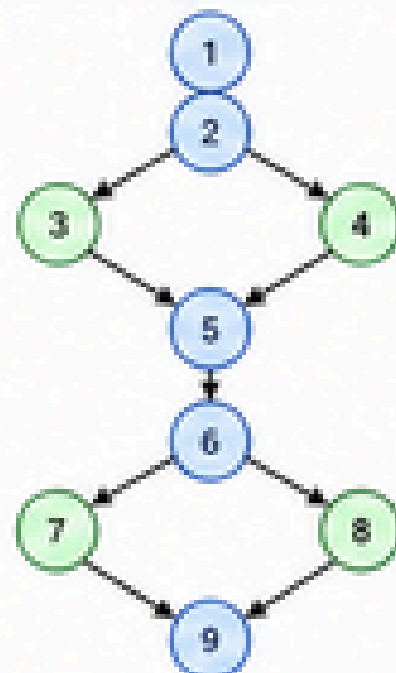


DILAKUKAN OLEH DEVELOPER / TESTER TEKNIS

CONTOH KODE

```
1  if (x > 0) {  
2      y = x * 2;  
3  } else {  
4      y = -x;  
5  }  
6  if (y > 10) {  
7      print("Besar");  
8  } else {  
9      print("Kecil");  
10 }  
11
```

FLOW GRAPH DARI KODE



KETERANGAN DARI INDEPENDEN

Path 1 : 1 - 2 - 3 - 5 - 6 - 7 - 9

Path 2 : 1 - 2 - 4 - 5 - 6 - 7 - 9

Path 3 : 1 - 2 - 4 - 5 - 6 - 8 - 9

Cyclomatic Complexity ($V(G)$) = $E - N + 2$

$E = 11$ (edge), $N = 9$ (node)

$V(G) = 11 - 9 + 2 = 4$

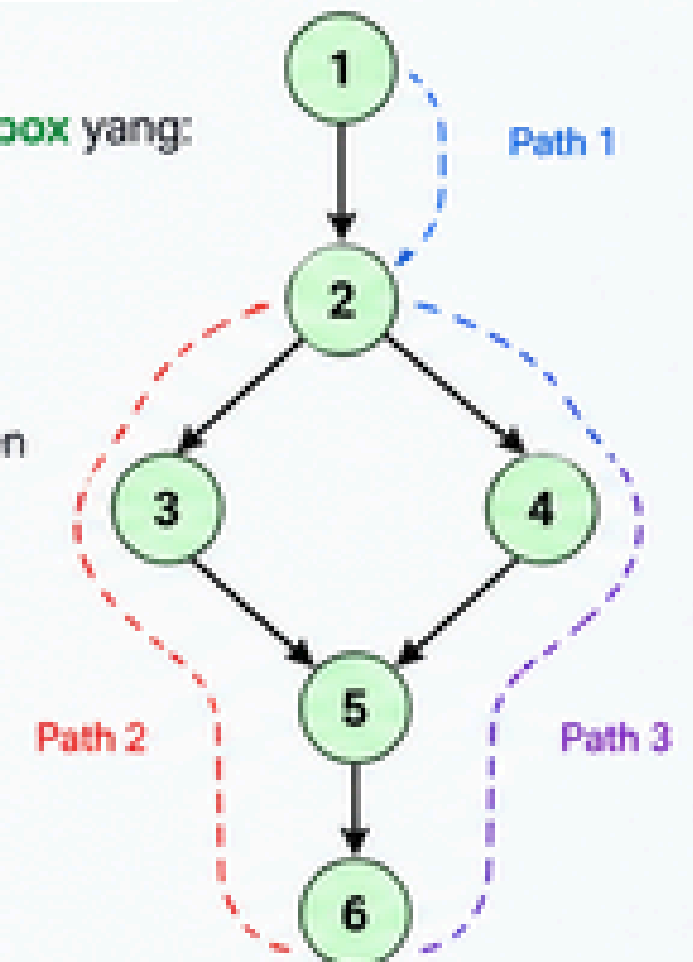
⇒ Jumlah path independen minimal = 4



APA ITU BASIS PATH TESTING?

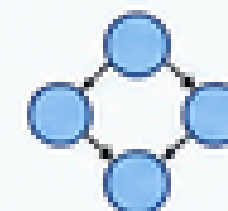
Basis Path Testing adalah teknik **white box** yang:

- ✓ Menggunakan flow graph
- ✓ Mengidentifikasi semua jalur independen
- ✓ Menjamin setiap statement dieksekusi minimal sekali



KONSEP UTAMA

FLOW GRAPH
(GRAF ALUR PROGRAM)



Representasi grafis dari alur kontrol program

NODE
(SIMPUL)



Titik atau lokasi yang mewakili satu atau lebih statement program

EDGE
(PANAH/ALUR)



Alur kontrol yang menghubungkan antar node

PATH
(JALUR EKSEKUSI)



Urutan node yang dilalui selama eksekusi program



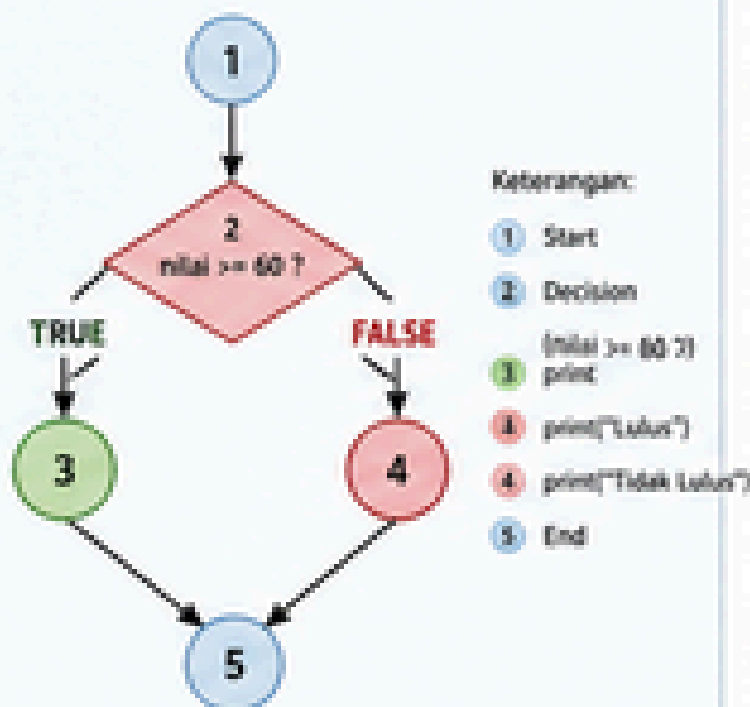
LANGKAH-LANGKAH BASIS PATH TESTING



1

BUAT FLOW GRAPH DARI PROGRAM

Flow graph merepresentasikan aliran kontrol program ke dalam bentuk graf.



2

HITUNG CYCLOMATIC COMPLEXITY

Rumus yang dapat digunakan:

$$V(G) = E - N + 2P$$

atau

$$V(G) = P + 1$$

E = Jumlah edge (panah)
N = Jumlah node (simpul)
P = Jumlah connected component (biasanya P = 1)

$$\begin{array}{l} E = 5 \\ N = 5 \\ P = 1 \end{array} \quad \begin{array}{l} V(G) = 5 - 5 + 2(1) \\ = 2 \end{array}$$

Cyclomatic Complexity = 2

3

TENTUKAN JUMLAH PATH INDEPENDEN

Jumlah path independen sama dengan nilai Cyclomatic Complexity.

Jumlah Path Independen
= $V(G) = 2$

Path independen adalah kumpulan path yang setiap path-nya memperkenalkan paling sedikit satu edge baru yang belum dilalui oleh path sebelumnya.

4

IDENTIFIKASI SEMUA PATH

Berdasarkan flow graph, identifikasi semua path yang mungkin.

Path 1 (kondisi TRUE)

1 → 2 → 3 → 5
(nilai >= 60 adalah TRUE)

Path 2 (kondisi FALSE)

1 → 2 → 4 → 5
(nilai >= 60 adalah FALSE)

★ Semua path yang ditemukan (2 path) sudah sama dengan jumlah path independen.

5

BUAT TEST CASE UNTUK SETIAP PATH

Buat minimal satu test case untuk mengeksekusi setiap path independen.

Path	Kondisi	Data Uji (nilai)	Expected Output
Path 1 (1-2-3-5)	nilai >= 60 (TRUE)	75 atau 60	Lulus
Path 2 (1-2-4-5)	nilai >= 60 (FALSE)	59 atau 0	Tidak Lulus

Catatan:

- Pastikan setiap path dieksekusi minimal 1 kali.
- Test case di atas adalah contoh, nilai lain yang memenuhi kondisi juga valid.

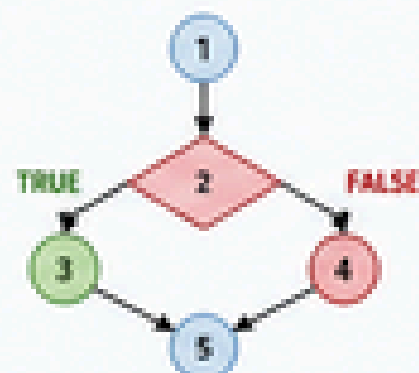


CONTOH STUDI KASUS

PSEUDOCODE

```
1 if (nilai >= 60)
2   print("Lulus")
3 else
4   print("Tidak Lulus")
```

FLOW GRAPH



RINGKASAN HASIL

- Cyclomatic Complexity ($V(G)$) = 2
- Jumlah Path Independen = 2
- Path yang Ditemukan = 2
- Test Case yang Dibuat = 2

PATH

- **Path 1** → 1 → 2 → 3 → 5 (kondisi TRUE)
- **Path 2** → 1 → 2 → 4 → 5 (kondisi FALSE)

TEST CASE

Path	Data Uji (nilai)	Expected Output
Path 1	75 (atau 60)	Lulus
Path 2	59 (atau 0)	Tidak Lulus

WHITE BOX TESTING

Pengujian yang dilakukan dengan mengetahui struktur internal kode/program seperti logic, struktur kontrol, dan alur eksekusi.

CONTOH CONTOH WHITE BOX TESTING

CIRI-CIRI

- ✓ Mengetahui struktur kode/program
- ✓ Menguji alur logika, decision, loop, dan path internal
- ✓ Biasanya dilakukan oleh developer atau tester teknis
- ✓ Contoh teknik: Statement Coverage, Branch Coverage, Condition Coverage, Path Coverage

KEUNTUNGAN

- ✓ Mendeteksi kesalahan pada logika dan alur program
- ✓ Meningkatkan kualitas kode dan performa
- ✓ Memastikan semua path kode tereksekusi

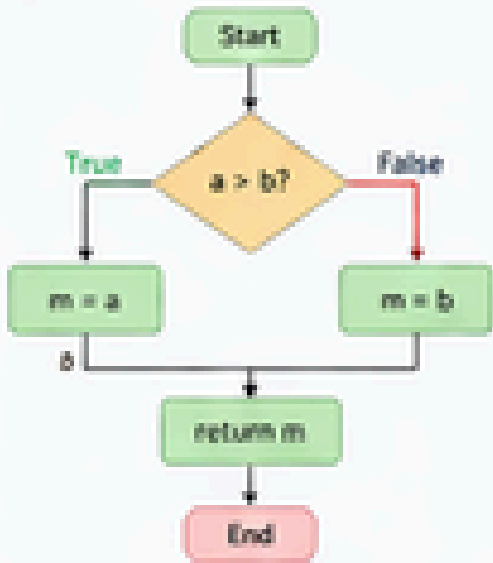
KETERBATASAN

Tidak memastikan apakah kebutuhan pengguna sudah terpenuhi (fokus pada kode, bukan fungsionalitas).

1 Statement Coverage

Tujuan: Memastikan setiap statement dieksekusi minimal sekali.

```
1 int max(int a, int b) {
2   int m;
3   if (a > b) {
4     m = a;
5   } else {
6     m = b;
7   }
8   return m;
9 }
```



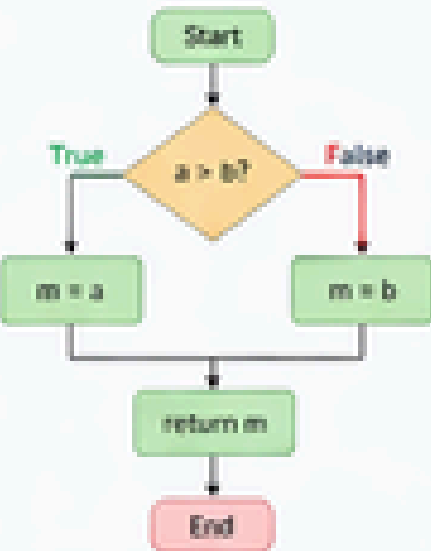
Test Case	Hasil
a = 5, b = 3	Semua statement tereksekusi
a = 2, b = 7	Semua statement tereksekusi

Minimal 2 test case untuk mengeksekusi seluruh statement (baris 2-8).

2 Branch Coverage

Tujuan: Memastikan setiap cabang (true/false) dieksekusi minimal sekali.

```
1 int max(int a, int b) {
2   int m;
3   if (a > b) {
4     m = a;
5   } else {
6     m = b;
7   }
8   return m;
9 }
```



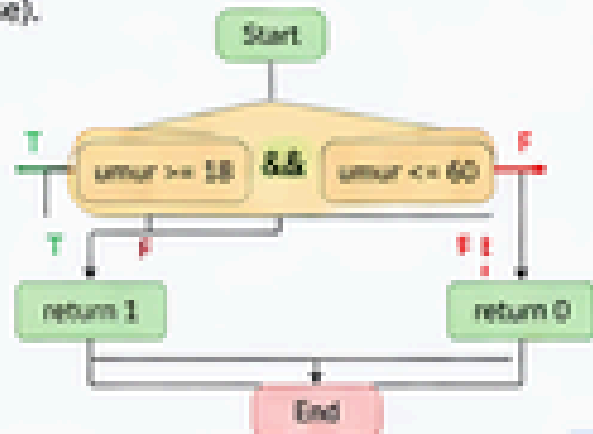
Test Case	Hasil
a = 5, b = 3	True branch
a = 2, b = 7	False branch

Setiap cabang (True dan False) harus dieksekusi minimal sekali.

3 Condition Coverage

Tujuan: Memastikan setiap kondisi (true/false) pada ekspresi logika dievaluasi minimal sekali (true dan false).

```
1 int cekDewasa(int umur) {
2   if (umur >= 18 && umur <= 60) {
3     return 1; // dewasa
4   } else {
5     return 0; // bukan dewasa
6   }
7 }
```



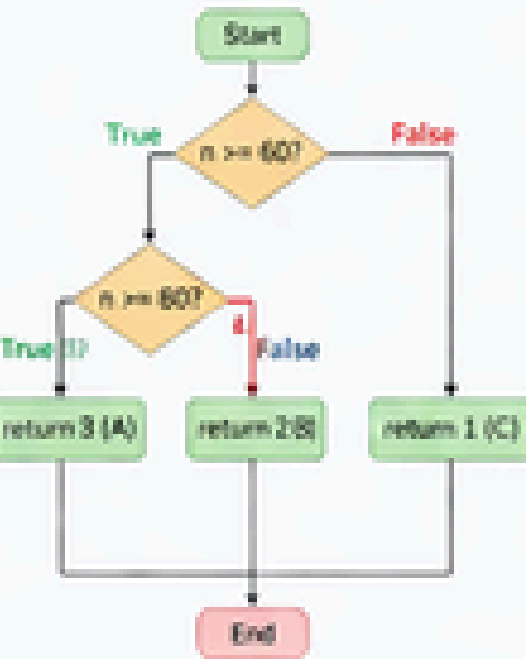
Test Case	umur >= 18	umur <= 60	Hasil
umur = 17	F	T	return 0
umur = 25	T	T	return 1
umur = 65	T	F	return 0

Semua kondisi (umur >= 18 dan umur <= 60) dievaluasi true dan false.

4 Path Coverage

Tujuan: Memastikan semua jalur independen pada program dieksekusi.

```
1 int nilai(int n) {
2   if (n >= 60) {
3     if (n >= 80) {
4       return 3; // A
5     } else {
6       return 2; // B
7     }
8   } else {
9     return 1; // C
10  }
11 }
```



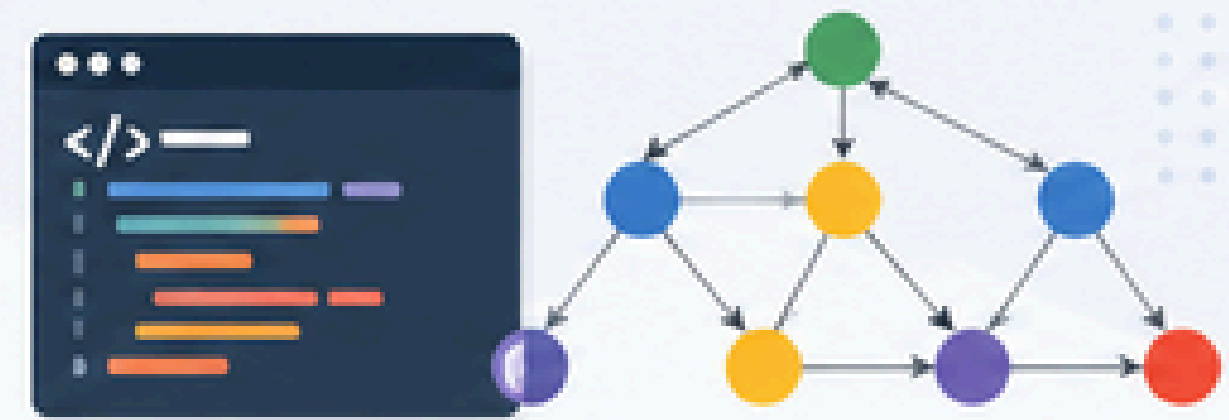
Path	Test Case Contoh
A: 60 <= n dan n >= 80	n = 85
B: 60 <= n dan n < 80	n = 70
C: n < 60	n = 40

Minimal 3 test case untuk mengeksekusi semua jalur independen (A, B, C).

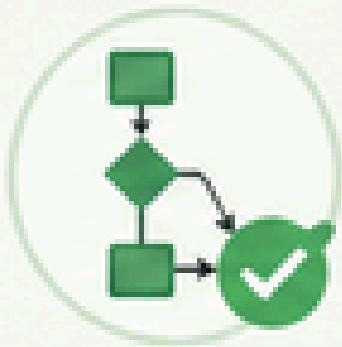
Catatan: White Box Testing berfokus pada struktur kode dan logika internal untuk memastikan setiap bagian kode bekerja sesuai desain.



KELEBIHAN & KEKURANGAN BASIS PATH TESTING



KELEBIHAN BASIS PATH TESTING



Menguji semua jalur logika program

Memastikan setiap jalur independen dieksekusi sehingga semua kemungkinan logika diuji.



Mengidentifikasi error logika

Efektif menemukan kesalahan pada struktur logika, kondisi, dan percabangan.



Memberikan coverage tinggi

Memberikan cakupan yang lebih menyeluruh dibanding banyak metode pengujian lainnya.

VS

KEKURANGAN BASIS PATH TESTING



Kompleks untuk program besar

Jumlah jalur independen meningkat secara eksponensial seiring bertambahnya ukuran dan kompleksitas program.



Mebutuhkan pemahaman kode

Tester harus memahami struktur dan logika kode dengan baik untuk menentukan jalur independen yang benar.



Flow graph bisa rumit

Flow graph untuk program besar dapat menjadi sangat kompleks dan sulit dianalisis.



Basis Path Testing sangat efektif untuk memastikan kualitas logika program, namun perlu dipertimbangkan kompleksitas dan sumber daya yang dibutuhkan.



Thank You!

See You Next Week